



Introduction To Python

Dr. Hnin Lai Nyo
Faculty of Computer Science
UCS(Taungoo)

Contents

- What is Python?
- History of Python
- Installation steps for Python Environment
- Basic Elements of Python
- Data Types
- Casting
- Operators
- Loops and Decision Making
- Summary

What is Python?

- Python is a high-level, general-purpose programming language.
- Its design philosophy emphasizes code readability with the use of significant indentation.
- It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming.
- It is often described as a "batteries included" language due to its comprehensive standard library.

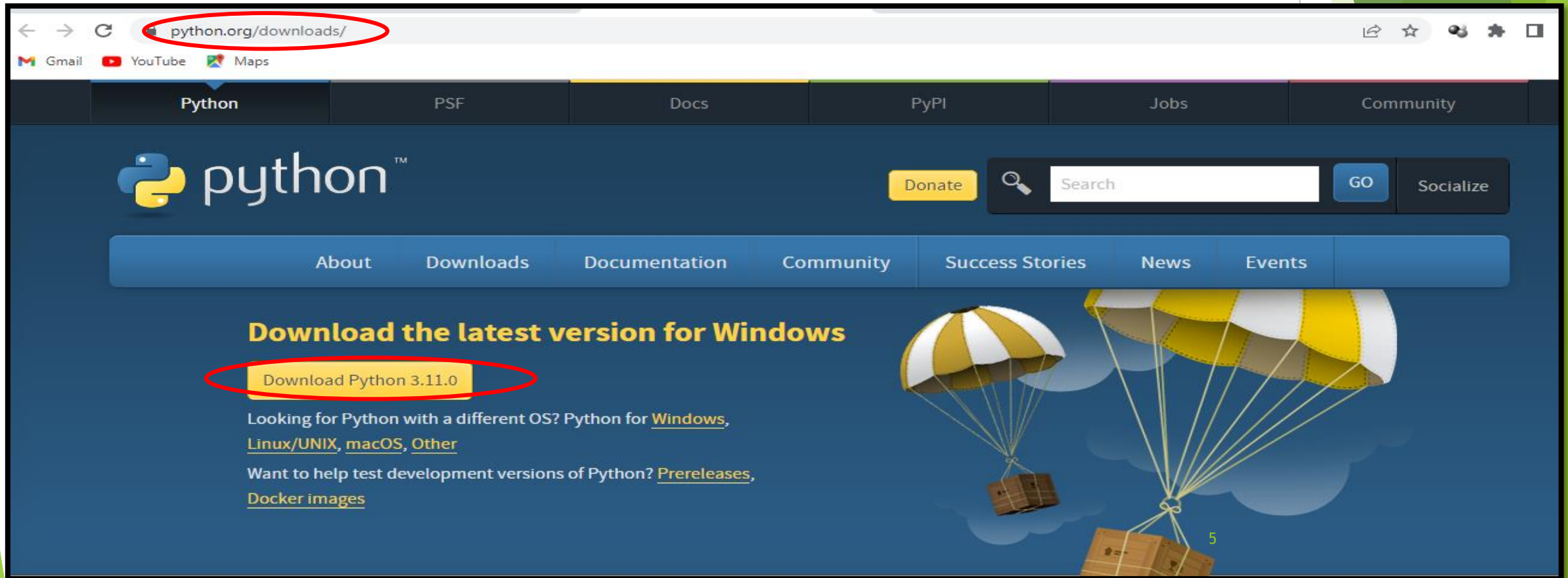
History of Python

- Guido van Rossum began working on Python in the late 1980s as a successor to the ABC programming language and first released it in 1991 as Python 0.9.0.
- Python 2.0 was released in 2000 and introduced new features such as list comprehensions, cycle-detecting, garbage collection, reference counting, and Unicode support.
- Python 3.0, released in 2008, was a major revision that is not completely backward-compatible with earlier versions.
- Python 2 was discontinued with version 2.7.18 in 2020.
- Python consistently ranks as one of the most popular programming languages.



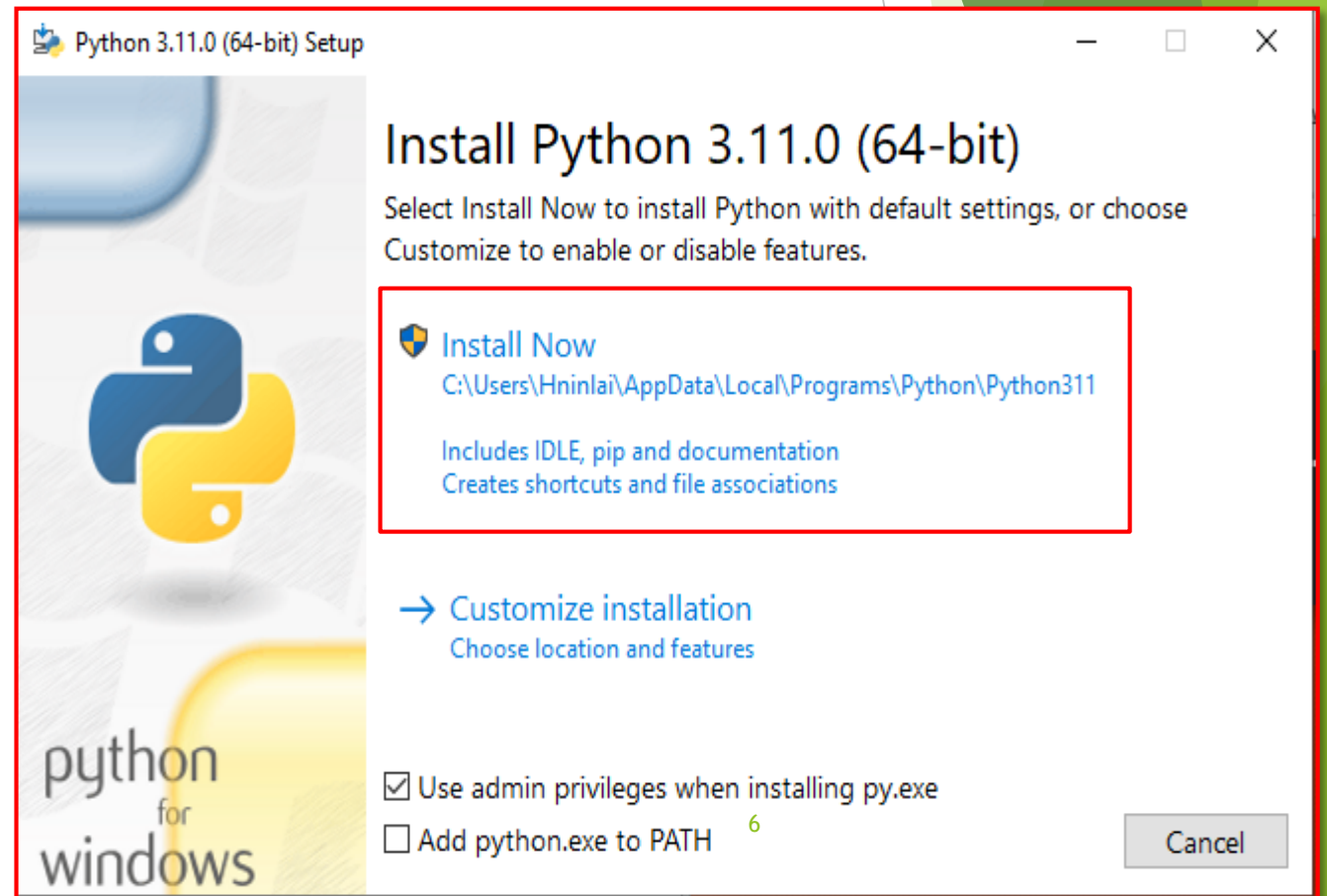
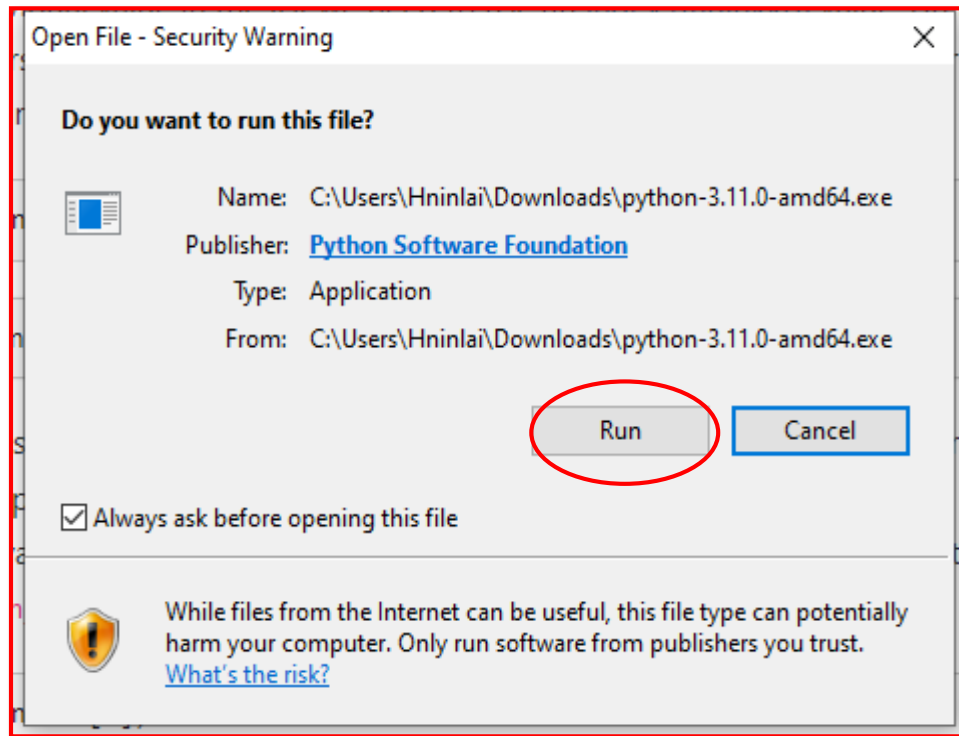
Installation steps for Python Environment

- Download Python latest version from www.python.org/downloads/ Windows x86-64 executable installer and run the .exe file.



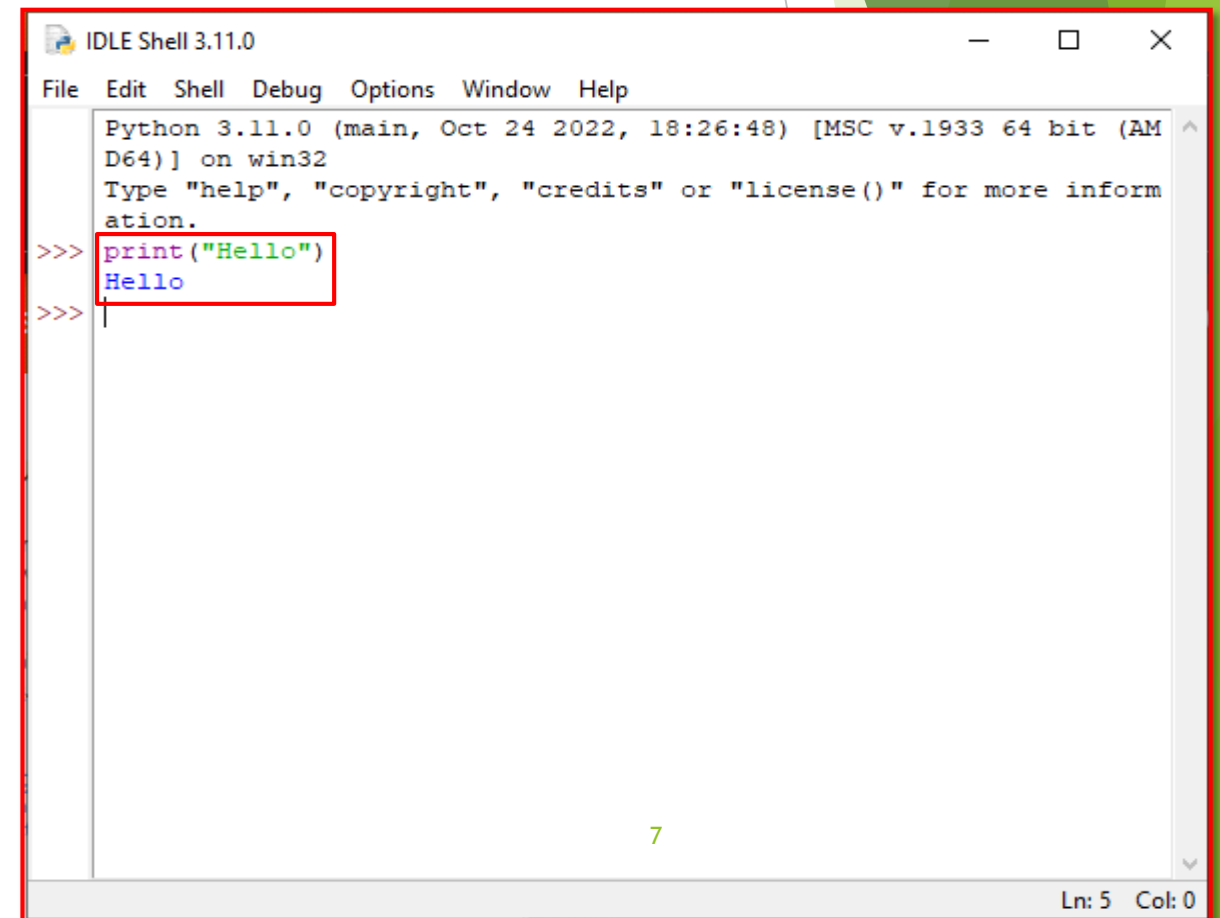
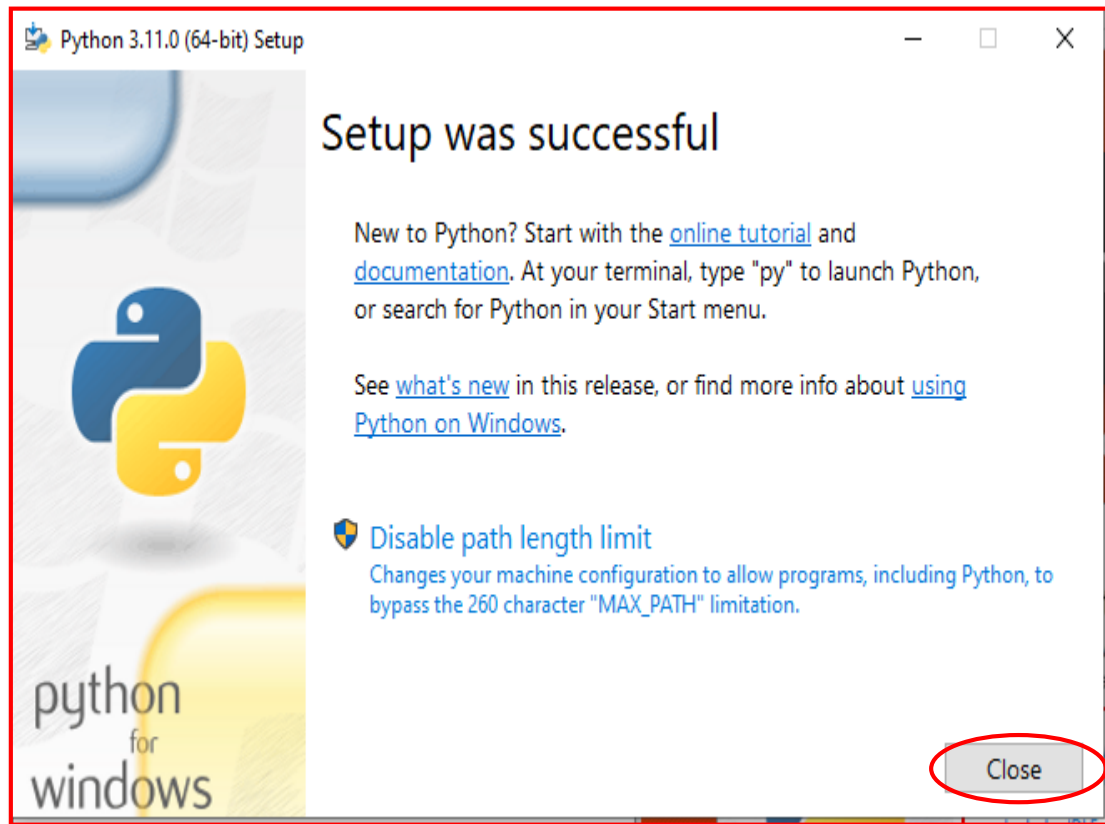
Installation steps for Python Environment

- Firstly, click the run button and then click Install Now option.



Installation steps for Python Environment

- If setup was successful, click close button. Then you can start to run python in IDLE shell.



Basic Elements of Python

Variable and Data Types

- Data types are nothing but variables you use to reserve some space in memory.
- Python variables do not need an explicit declaration to reserve memory space.
- To create a variable in Python, all you need to do is specify the variable name, and then assign a value to it.
- **<variable name> = <value>**
- E.g.;

a = 5	➔ a is variable and int type
b = 3.14	➔ b is variable and float type
name = "MgMg"	➔ name is variable and str type
x = None	➔ x is variable and None type

Basic Element of Python

Indentation

- For python, all of the grouping of statements are written on indentations.

- **Example:**

```
def separatefun(b):  
    for i in b:  
        if i== 1:  
            return True  
    return False
```

```
separatefun([2,3,5,6,1])
```

Basic Element of Python

Comments

- Comments are used to explain code when the basic code itself isn't clear
- Python ignores comments, and it will not execute code
- Single line comments begin with the hash character (#) and are terminated by the end of line.

❖ **Examples:** Single line comment

```
print("Hello! My Dear")    # This line prints "Hello! My Dear"
```

❖ **Examples:** Multiple lines comment

```
“ “ ”
```

This type of comment spans multiple lines.

These are mostly used for documentation of functions, classes and modules

```
“ “ ”
```

Basic Elements of Python

Reserved words

- Reserved words can not be used as variable names (identifiers).

Keywords In Python

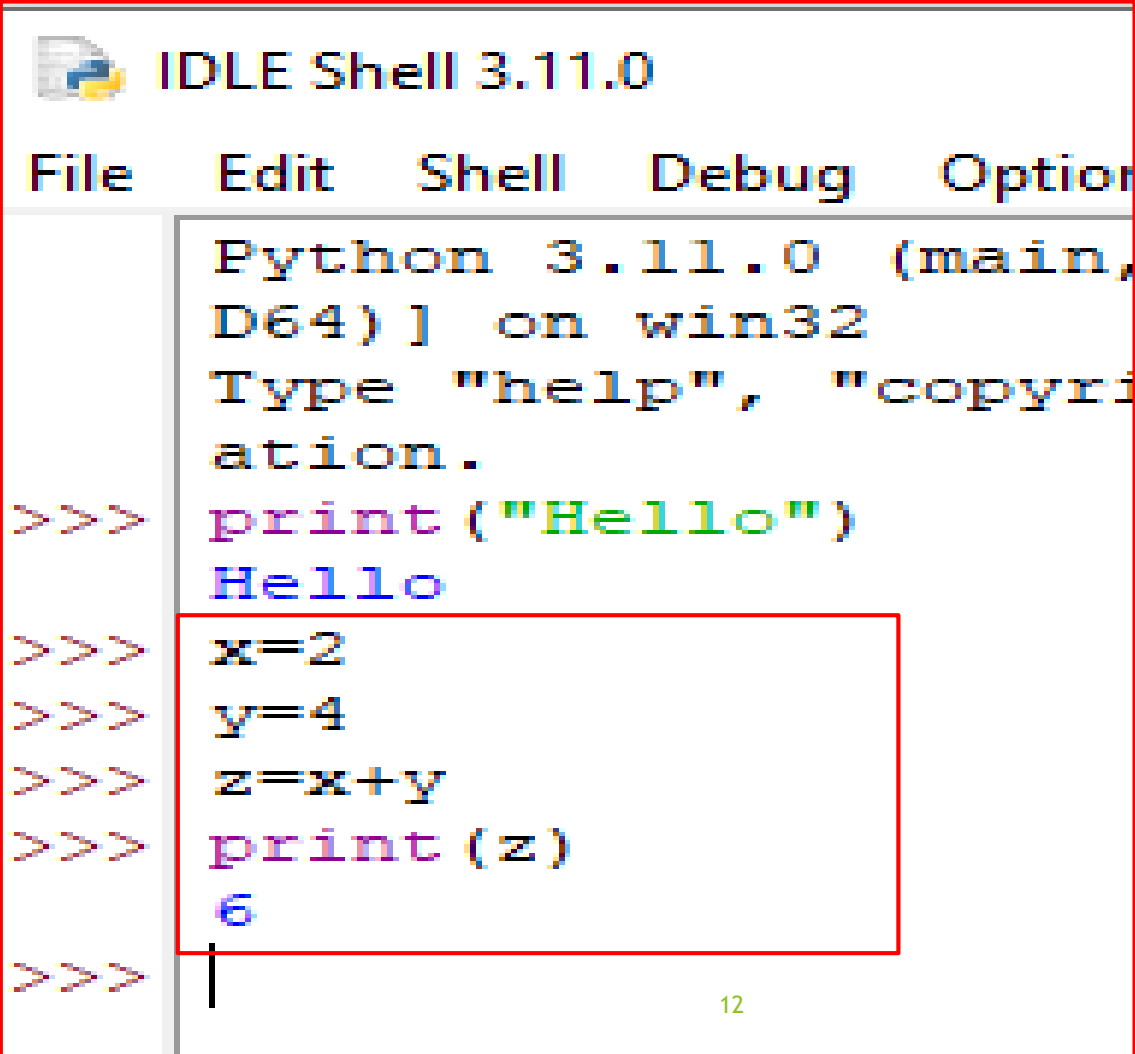
- | | | |
|------------|-----------|------------|
| • and | • False | • nonlocal |
| • as | • finally | • not |
| • assert | • for | • or |
| • break | • from | • pass |
| • class | • global | • raise |
| • continue | • if | • return |
| • def | • import | • True |
| • del | • in | • try |
| • elif | • is | • while |
| • else | • lambda | • with |
| • except | • None | • yield |

Basic Elements of Python

Sentences Structure

```
x=2
y=4
z = x+y
print(z)
```

x,y,z (variables)
+,= (operators)
print() (function)



The screenshot shows the IDLE Shell 3.11.0 interface. The menu bar includes File, Edit, Shell, Debug, and Options. The shell window displays the following text:

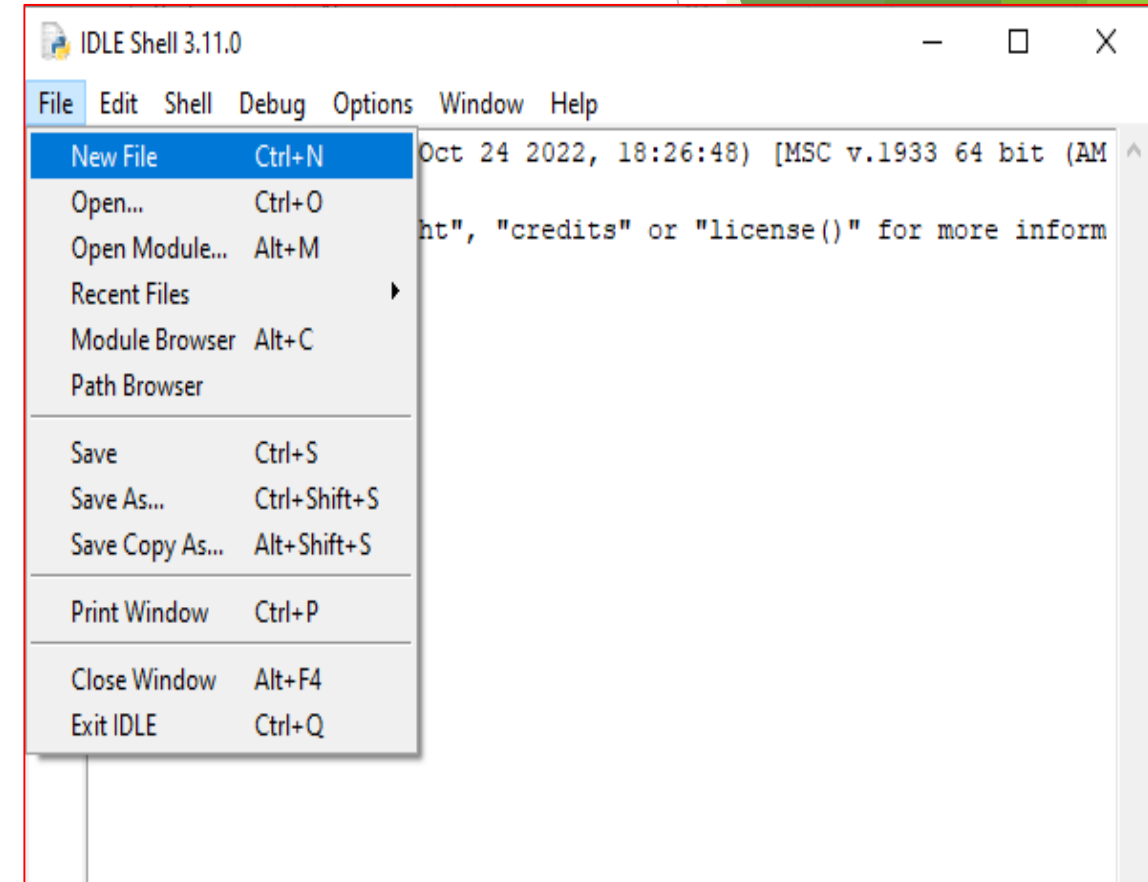
```
Python 3.11.0 (main,
D64) ] on win32
Type "help", "copyr
ation.
>>> print("Hello")
Hello
>>> x=2
>>> y=4
>>> z=x+y
>>> print(z)
6
>>> |
```

A red rectangular box highlights the code block from `x=2` to `print(z)` and its output `6`.

Basic Elements of Python

Story Structure (Writing all codes in a .py file)

- Most of the program can contain many sentences to execute, so we have to write them into a file and save that file with .py extension.
- To create .py file, select New file from File menu in IDLE shell and then click save button in file menu of newfile with filename.py.



Basic Elements of Python

Example: Factorial

```
num = int (input ("Enter a number : "))
factorial = 1
i=1
if num<0:
    print("No factorial exist for negative numbers")
elif num==0:
    print ("The factorial is 1")
else:
    while i<=num:
        factorial = factorial * i
        i = i+1
    print("The factorial of ", num, "is", factorial)
```

Data Types

Built-in Data Types

- Text Type : str
- Numeric Type : int, float, complex
- Sequence Type : list, tuple, range
- Mapping Type : dict
- Set Type : set, frozenset
- Boolean Type : bool
- Binary Type : bytes, bytearray, memoryview
- None Type : None Type

Data Types

String Data Type

- String are identified as a contiguous set of characters represented in the quotation marks.
- Python allows for either pairs of single or double quotes.
- **Example:**

```
a_str = 'Hello World'  
print (a_str)  
print(a_str[0])  
print (a_str[0:5])
```

Output:

```
Hello World  
H  
Hello
```

Data Types

String Data Type

- Concatenation: "hello" + "world" → "helloworld"
- Repetition: "hello" * 3 → "hellohellohello"
- Indexing: "hello"[0] → "h"
"hello"[-1] → "o"
- Slicing: "hello"[1:4] → "ell"
- Comparison: "hello" < "jello" → True
- Search: "e" in "hello" → True

Data Types

String Data Type

- `a = " Hello World "`
- `len()` => To get the length of a string
`print(len(a)) => 13`
- `strip()` => To remove any whitespace from the beginning or the end.
`print(a.strip()) => Hello World`
- `lower()` => To returns the string in lower case.
`print(a.lower()) => hello world`
- `upper()` => To returns the string in upper case.
`print(a.upper()) => HELLO WORLD`
- `replace()` => To replaces a string with another string
`print(a.replace("World", "Earth")) => Hello Earth`

Data Types

String Data Type

- `a = " Hello World "`
- `split()` => To split the string into substrings if it finds instances of the separator.
`print(a.split(" ")) => ['Hello', 'World']`
- `isnumeric()` => To checks whether the string consists of only numeric characters.
`print(a.isnumeric()) => False`
- `isdigit()` => To checks whether the string consists of digits only
- `isalpha()` => To checks whether the string consists of alphabetic characters only.
- `format()` => To combine strings and numbers
`age=22`
`txt="My name is Mary, and I am {}"`
- `Print(txt.format(age)) => My name is Mary, and I am 22`

Data Types

Numeric Data Type

- Numbers have four types in python (int, float, complex, long).

❖ Examples

a = 10 (int)

b = 10.2 (float)

c = 3.14j (complex)

d = 1234567L (long)

- Type() function is used to verify the type of any object in python.

❖ Example

x = 1

print(type(x))

Output:

<class 'int'>

Data Types

List Data Type

- A list contains items separated by commas and enclosed within square bracket [].
- List are more similar to array in C and one difference is that all the items belonging to a list can be of different data type.

❖ Examples:

```
list = [123, 'abcd', 10.2 , 'd']
```

```
list1 = ['cat', 'rat']
```

```
print(list)
```

```
print (list[0:2])
```

```
print (list1 * 2)
```

```
print (list + list1)
```

Output:

```
[123, 'abcd', 10.2, 'd']
```

```
[123, 'abcd']
```

```
['cat', 'rat', 'cat', 'rat']
```

```
[123, 'abcd', 10.2 , 'd', 'cat', 'rat']
```

Data Types

List Data Type

- **list()** constructor => To create a list
thislist= list(('flower','fruit','food')) => **thislist = ['flower', 'fruit', 'food']**
- **len()** => To know the length of the list
- **append()** => To add an item to the end of the list
thislist.append('fluid')
print(thislist) => **['flower','fruit','food','fluid']**
- **insert()** => To add an item at the specified index
thislist.insert(0,'flavour')
print(thislist) => **['flavour','flower','fruit','food','fluid']**
- **remove()** => To remove the specified item
thislist.remove('fruit')

Data Types

List Data Type

- **pop()** => To removes the specified index or last item (if no index)
- **del** keyword => To removes the specified index or completely list
 `del thislist[0]`
 `del thislist`
- **clear()** => To do empty the list
 `thislist.clear( )`
 `print(thislist) => [ ]`
- **copy()** => To make a copy of list
 `copylist=thislist.copy()`
 `print(copylist) => [ ]`
- **remove()** => To removes the specified item
 `thislist.remove('fruit')`

Data Types

Tuple Data Type

- Lists are enclosed in brackets [] and their elements and size can be changed, while tuples are enclosed in parentheses () and cannot be updated.

- **Examples:**

```
tuple = (123, 'hello')
```

```
tuple1 = (123, 'world')
```

```
tup = "a", "b", "c", "d"
```

```
tup1 = ('orange',)
```

```
print (tuple)
```

```
print (tuple[0])
```

```
tuple[1]= 'update'
```

Output:

```
(123, 'hello')  
(123)  
error
```

Data Types

Dictionary Data Type

- Dictionary consists of key-valued pairs.
- It is enclosed by curly braces { } and valued can be assigned and accessed using square brackets [].

❖ Example:

```
dic = {'name' : 'red', 'age': 10 }  
dd = dict(name:'Mg Mg', age: 20} // constructor  
print (dic)  
print( dic['name'])  
print(dic.values())  
print(dic.keys())
```

Output:

```
{'name': 'red', 'age': 10}  
red  
{'red',10}  
['name', 'age']
```

Data Types

Set Data Type

- Sets are unordered collections of unique objects.
- **Sets** : They are mutable and new elements can be added once sets are defined.

❖ Examples:

```
basket = { 'apple', 'orange', 'apple', 'pear', 'orange', 'banana' }
```

```
print (basket)
```

```
a = set('abracadabra')
```

```
print (a)
```

```
a.add('z')
```

```
print (a)
```

Output:

```
{'orange', 'banana', 'pear', 'apple'}
```

```
{'a', 'r', 'b', 'c', 'd'}
```

```
{'a', 'c', 'r', 'b', 'z', 'd'}
```

Data Types

Frozen Data Type

- **Frozen Sets** : They are mutable and new elements cannot added after it defined.

- ❖ **Example:**

```
b = frozenset('asdfagsa')
```

```
print (b)
```

```
cities = frozenset( ["Frankfurt", "Basel", "Freiburg"])
```

```
print (cities)
```

Output:

```
frozenset( {'f', 'g', 'd', 'a', 's'})
```

```
frozenset({'Frankfurt', 'Basel', 'Freiburg'})
```

Data Types

Boolean Type

- Booleans represent one of two values: True or False.

❖ Examples:

```
b= True
```

```
Print(b)
```

```
c=False
```

```
Print(b,c)
```

Output:

True

True False

Data Types

Binary Types are bytes, bytearray, memoryview

- **Examples:**

```
x= bytes(5)
```

```
print(x)
```

```
x=bytearray(5)
```

```
print(x)
```

```
x=memoryview(bytes(5))
```

```
print(x)
```

Output:

```
b'\x00\x00\x00\x00\x00'
```

```
bytearray(b'\x00\x00\x00\x00\x00')
```

```
<memory at 0x000001FF13710C40>
```

Type Conversion

- We can convert from one type to another with `int()`, `float()` and `complex()` methods.

- ❖ **Examples:**

`x = 1` $\rightarrow$ int

`y = 2.8` $\rightarrow$ float

`z = 1j` $\rightarrow$ complex

`a = float(x)`

`b = int(y)`

`c = complex(x)`

`print(type(a)) ?`

Operators

- Operators are used to perform operations on variables and values.
- Python divides the operators in the following groups
 - ❖ Arithmetic operators
 - ❖ Assignment operators
 - ❖ Comparison operators
 - ❖ Logical operators
 - ❖ Identity operators
 - ❖ Membership operators
 - ❖ Bitwise operators

Operators

Arithmetic Operators

- Arithmetic operators are used with numeric values to perform common mathematical operation.
- The **math** module offers expanded functionality like trigonometric functions, root operations, logarithms, and many more.

>>> a,b = 1, 2

>>> import **operator**

>>> a+b (or) >>> operator.**add**(a,b)

>>> operator.**sub**(a,b)

>>> operator.**mul**(a,b)

>>> operator.**divmod**(a,b)

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Operators

Assignment Operators

- Assignment operators are used to assign values to variables.

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3

Operator	Example	Same as
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Operators

Comparison Operators

- Comparison operators are used to compare two values.

Operator	Name	Example
==	Equal	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

Operators

Logical Operators

- Comparison operators are used to combine conditional statements.

Operator	Description	Example
and	Returns true if both statements are true	<code>x <5 and x<10</code>
or	Returns true if one of the statements are true	<code>x <5 or x<4</code>
not	Returns the result, returns False if the result is true	<code>not (x<5 and x<10)</code>

Operators

Identity Operators

- Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns true if both variables are the same object	x is y
is not	Returns true if both variables are not the same object	x is not y

Example:

```
x=["apple", "orange"]  
y=["apple", "orange"]  
z=x  
print(x is z)  => True  
print(x is y)  => False  
print(x == y) => True
```

Operators

Membership Operators

- Membership operators are used to test if a sequence is presented in an object.

Operator	Description	Example
in	Returns true if a sequence with specified value is presented in the object	x in y
not in	Returns true if a sequence with the specified value is not present in the object	x not in y

Example:

```
x=["apple", "orange"]
```

```
print("apple" in x)      => True
print("grape" in x)     => False
print("grape" not in x) => True
```

Operators

Bitwise Operators

- Bitwise operators are used to compare (binary) numbers.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits its 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bit fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Operators

Operator Precedence

- Highest precedence at top,
- Lowest at the bottom.
- Same level operators evaluate left to right.

Operator	Description
()	Parentheses (grouping)
f (args...)	Function call
x [index : index]	Slicing
x [index]	Subscription
x.attribute	Attribute reference
**	Exponentiation
~x	Bitwise not
+x , -x	Positive, negative
*, / , %	Multiplication, division. Remainder
+, -	Addition, subtraction
<< , >>	Bitwise shifts
&	Bitwise and
^	Bitwise xor
	Bitwise or

Exercises

- <https://pynative.com/python-operators-and-expression-quiz/>
- [**https://pynative.com/python-variables-and-data-types-quiz/**](https://pynative.com/python-variables-and-data-types-quiz/)
- <https://pynative.com/basic-python-quiz-for-beginners/>
- <https://pynative.com/python/quizzes/>
- <https://pynative.com/python-basic-exercise-for-beginners/>

Thank You